关于在ZYNQ7020上部署TINYYOLO
实现机场鸟类检测模拟的报告

摘要

针对机场鸟类检测模拟中计算资源受限和卷积推理耗时较长的问题，本文基于Zynq-7020设计了PS与PL协同的目标检测与追踪原型。系统采用192×192输入，将改进TinyYOLO模型的24层卷积部署于PL，其中22层采用INT8计算与INT32累加，两个末端分类卷积保留参考实现的浮点运算行为。计算单元结合双INT8打包乘法和15级点积流水线，并通过片上输入缓存、分组权重复用、激活块跳零、算子融合及写合并减少计算和访存开销；PS负责图像预处理、未融合的图算子、检测后处理及基于去均值SAD的模板追踪，并集成双轴云台控制接口。通过计算与控制路径流水化、高扇出优化及修复时钟缓冲连接，PL在240MHz下实现时序收敛。板级验证中，标准输入热帧推理耗时727ms，约1.38帧/秒，三组输入的150项图步骤哈希均与参考结果一致，其中包含72项卷积哈希；工程使用216个DSP和107个BRAM块。该原型形成了从模型部署、RTL优化到板级数值验证的完整工程链路，为资源受限FPGA上的视觉检测应用提供实现参考。

关键词： Zynq-7020；鸟类检测；FPGA加速；混合精度；数据流优化

目录

第一部分_作品概述 3
1.1功能与特性 3
1.2应用领域 3
1.3主要技术特点 3
1.4主要性能指标 4
1.5主要创新点 5
1.6设计流程 6
第二部分_系统组成及功能说明 7
2.1整体介绍 7
2.2硬件系统介绍 7
2.2.1硬件整体介绍 7
2.2.2机械结构介绍 8
2.2.3电路模块介绍 10
2.3软件系统介绍 13
2.3.1软件整体介绍 13
2.3.2软件各模块介绍 13
第三部分_完成情况及性能参数 14
3.1整体介绍 14
3.2工程成果 15
3.2.1机械成果 16
3.2.2电路成果 16
3.2.3软件成果 16
3.3特性成果 16
第四部分_总结 18
4.1可拓展处 18
4.2心得体会 18
第五部分_参考文献 20

第一部分_作品概述

1.1功能与特性

使用xc7z020clg484-1 ZYNQ7020开发板，部署Tinyyolo3yu，连接云台舵机与AR0234摄像模块，初步实现对鸟类的初步追踪检测与估算模拟飞行数据。通过极低成本的检测，为高成本的实时监测提供改善思路与基础。

1.2应用领域

机场鸟类识别对机场安全特别重要。鸟击被国际航空联合会列为了“A”类航空灾难，哪怕是一只小麻雀，若与飞机相撞，也会造成极大的冲击力，极易引起发动机故障或机身破损等严重飞行事故。而且，近几年随着我们对环境保护越发重视，机场周围的生态环境肉眼可见的有了极大改善，防范难度越来越大。我们需要更加精准，更加高效的识别技术来帮助我们处理好这一人与自然和谐相处的问题。本项目利用FPGA的并行计算和低功耗优势，可以**运用于鸟类识别的实际初步检测与模拟**，提供改善思路与方案，**以极低成本的方案来探路，为高成本的实际处理需求打下基础**。

1.3主要技术特点

本方案采用Zynq-7020的PS与PL异构协同架构。PL承担24层卷积计算，其中22层使用INT8输入与权重、INT32累加，两个末端分类卷积保留参考实现的浮点运算行为；通过双INT8打包乘法、15级点积流水线和控制路径流水化，将PL工作频率提升至240 MHz并完成时序收敛。模型部署采用192×192输入，结合激活块跳零、算子融合、片上输入缓存、分组权重复用和64位写合并减少计算及DDR访问开销。PS负责图像预处理、未融合图算子和检测后处理，并运行去均值SAD多尺度模板追踪，通过历史帧回放处理检测延迟。

1.4主要性能指标

指标	当前结果	说明
FPGA 平台	XC7Z020-CLG484-1	Zynq-7000 PS＋PL 架构
网络输入	192×192 RGB	INT8 输入，共 110,592 字节
卷积硬件化	24 层卷积全部在 PL 执行	不代表整个计算图的所有算子均在 PL
计算精度	22 层 INT8，INT32 累加；两个精确浮点检测头	浮点头保持 FP64 累加及最终 FP32 转换行为
并行结构	16 个输出通道并行，每路 24 项点积	使用双 INT8 打包乘法
PL 工作频率	240 MHz	最终布线后实现及板端配置一致
标准输入热帧推理	727 ms/帧，约 1.38 FPS	最终固件生产推理实测
摄像头输入网络耗时	平均 693.9 ms/帧	10 帧均完成推理；不含完整采集和保存成本
摄像头完整流程	0.806 FPS	包含输入图像及叠框保存
异步摄像头流程	检测约 1.141 FPS，采集循环约 2.282 FPS	本次为无目标场景，不能作为有效追踪帧率
输入缓存	128 KiB	分银行同步 BRAM
分组权重存储	192 KiB	按输出通道组织，跨像素复用
DDR 接口	64 位 HP0，64 字节缓存行	支持 1～8 beat 突发访问
DMA 缓冲区	32 MiB	板端运行时实读配置
检测网格	12×12＋6×6，共 180 个位置	输出为 900 个 FP32 值
最终建立时序	WNS +0.067 ns，TNS 0	原始 240 MHz 约束下签核
最终保持时序	WHS +0.020 ns，THS 0	无保持违例
脉宽余量	+0.833 ns	最终报告结果
时钟拓扑	总 BUFG 3 个，无级联	PL BUFG 由 MMCM CLKOUT0 直接驱动
数值一致性	150 个图步骤哈希、72 个卷积哈希全部匹配	bird、random、bird_repeat 三组输入
生产推理验证	8 次全部通过	最终 ClockFix 固件

计算阵列在每拍满载条件下的理论峰值为：

```
16 × 24 × 240 MHz = 92.16 GMAC/s
```

按一次乘法、一次加法分别计数，对应 184.32 GOPS

1.5主要创新点

1. **双 INT8 打包，提高单颗 DSP 的计算资源效率。**

   在 `tinyyolo_dot24_pair.v` 中，让两个输出通道共用激活，将两份有符号权重打包到一次 DSP 乘法，通过符号修正拆出两份乘积。配合 16 输出通道、24 项点积并行结构，在 Zynq-7020 的资源限制内构建较大的整数计算阵列。

   最终 DSP 分配为 192 个打包乘法 DSP、16 个累加 DSP、8 个重数量化 DSP，合计 216 个。这样做可以提高计算资源效率。

2. **混合精度架构，兼顾整数计算效率和参考结果一致性。**

   主体卷积使用 INT8，减少数据存储与计算开销；`tinyyolo_requant36_stream.v` 保留完整有效乘积，流水实现移位、ties-to-even 舍入及饱和。两个浮点检测头通过 `tinyyolo_head_exact.v` 迁入 PL，维持既定浮点累加顺序和输出转换行为。

   优势是将数值一致性贯穿硬件优化过程，能够从小算子推进到逐层整网验证，当前哈希比较支持所测输入的数值一致性。

3. **缓存、权重复用与算子融合协同减少 DDR 访问。**

   `tinyyolo_hp0_burst_cache.v` 提供分银行输入缓存和突发填充；权重按输出通道分组，在多个像素之间重复使用。累加后衔接重数量化、激活、可选池化，并通过带字节掩码的 64 位合并写回减少中间结果访存。

   这种设计同时改善计算单元的数据供给和片外带宽利用，适合资源及带宽受限的嵌入式平台。

4. **按激活块跳零，利用运行时稀疏性。**

   `tinyyolo_allconv_control.v` 以 24 个激活为一组，整组为数值 0 时跳过相应 MAC，并通过块编号映射保持权重寻址正确。逐字节累计非零标志，避免在块末尾增加宽组合归约路径。

   这样就可以无需重新训练或剪枝模型，可以减少全零块的计算开销；实际收益随不同层和不同输入的激活稀疏程度变化。

5. **面向物理实现的流水线与控制路径协同优化。**

   计算侧使用 DSP 内部寄存器和分段加法树；控制侧处理 ready/valid 背压、宽地址进位、BRAM 输出、高扇出使能与复位分配。`tinyyolo_ctrl_hp0_pipe2.v` 通过带 skid 缓冲的流水切开控制器与 HP0 的长路径，保持背压下事务正确。

   最终在 240 MHz 下完成建立和保持时序收敛，同时修复级联 BUFG 问题。这体现了从 RTL 功能验证到布局布线、时钟连接和板级验证的完整实现能力。

6. **通过历史帧回放，处理慢检测结果与当前画面的时间差。**

   PL 执行卷积推理，ARM 执行模板追踪与控制。`camera_infer.py` 采用单后台检测 worker，最多保留一份在途推理，并保存带帧号的有界历史。检测返回后，在对应旧帧初始化模板，再逐帧推进到当前画面；过期或历史不连续的结果拒绝应用。

   `tracker_core.c` 使用 32×32 模板、去均值 SAD、三尺度搜索及粗搜后细搜。`target_control.py` 已集成双轴 PWM、死区、限幅、丢失目标停机和独立 watchdog。

1.6设计流程

设计流程如下：

选题 --> 边缘目标检测部署 --> 确定YOLO模型 --> 分析实际应用 --> 确定机场鸟类识别场景

方案1：YOLOV8方案-->仿真分析失败，资源不够，且功率超出预期

方案2：LENET5方案--> 受限于2018.3版本中blockdesign中时序与频率改变无法有效解决，无法有效进行设计

方案3：切换2025.2重新设计，采用更合适的Tinyyolo3yu模型

TinyYOLO3yu --> 模型训练(4155张train，819张val) --> 导出并验证640×640的FP32ONNX --> 评估输入尺寸并降到416×416

--> 导出并锁定416×416的FP32ONNX --> 解决ONNX量化流程中的兼容性问题 --> 尝试全卷积对称INT8量化 --> 改用激活非对称权重对称的INT8量化 --> 进一步压缩需要ARM执行的浮点计算 --> 扫描置信度阈值，锁定最终量化候选 --> 接入ARO234进行跑通和逻辑推理 --> 验证数据传输并进行冒烟测试，结合ROSS优化代码(循环) --> 上板测试，发现单帧过慢，高达12分钟 --> 考虑加速处理 --> 1.设计层面/2.模型层面.

1.设计层面
消除软件中的重复工作 --> 优化硬件调度,让INT32写回与重新量化重叠执行,减少串行等待 --> 连接开发板，根据真实访存数据扩大缓存(依然很慢) --> 分析布线限制，把优化重点放到控制和访存流水线
--> 优化到13s每帧 --> 考虑超频 --> 使用两级流水线解决setup报错，发现实际变慢
--> 扩大PL的计算覆盖率，并减少中间数据搬运，将原由ARM执行的两个浮点末端分类卷积迁入PL，实现24个卷积算子的全部硬件化；其余图算子及检测后处理由ARM协同执行。

2.模型层面
融合LUT、池化等步骤，减少中间特征图落到DDR-->合并INT8写回，在生产模式关闭不必要的INT32中间结果写回 --> 增加片上输入复用，跳过能够确认乘积为零的计算.

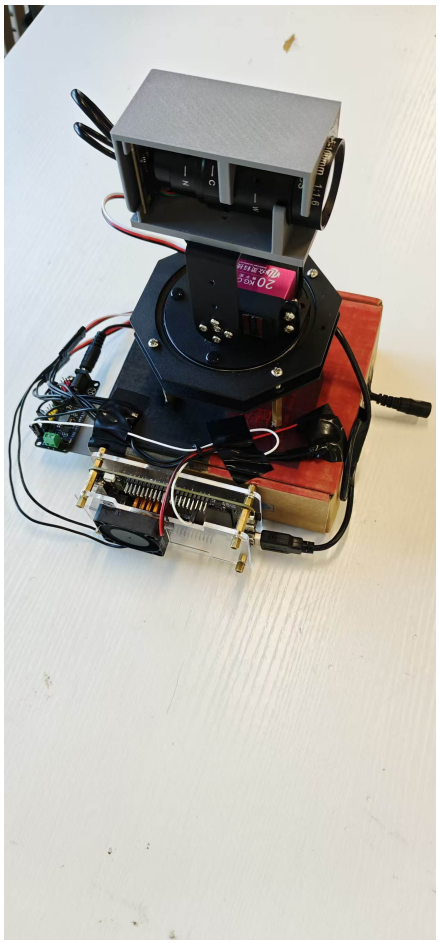
最后，补齐完整模型，用整网测试比较优化组合 --> 进一步提频，加大DSP并行量，将网络输入分辨率调整为192×192，并同步更新张量尺寸、检测网格与解码参数--> 优化至1.38FPS

优化后上板进行板级测试 --> 通过完整摄像头采集处理过程后开始设计云台的追踪控制

第二部分_系统组成及功能说明

2.1整体介绍

系统以云台为主，整个机械结构以箱子作为底座，使得云台结构晃动较小。板卡和其他模块放在箱子后方，控制云台舵机并且收集AR0234的图像数据，进行检测处理后，发送计算数据，实现简单估算和预测轨迹，跟踪鸟类，通过JTAG连接PC端后，也可以向PC端传输预测信息



系统整体画面

2.2硬件系统介绍

2.2.1硬件整体介绍

硬件系统可分为4部分：

1.两部分电源及稳压模块；2.云台舵机部分；3.AR0234摄像部分；4.FPGA板卡

PC → USB-JTAG桥 → Zynq JTAG：下载、配置和调试

PC ↔ USB-UART桥 ↔ PS UART：控制台或已实现的结果通信协议

电源 → 板卡供电输入：依据SP2原理图与实际接线注明

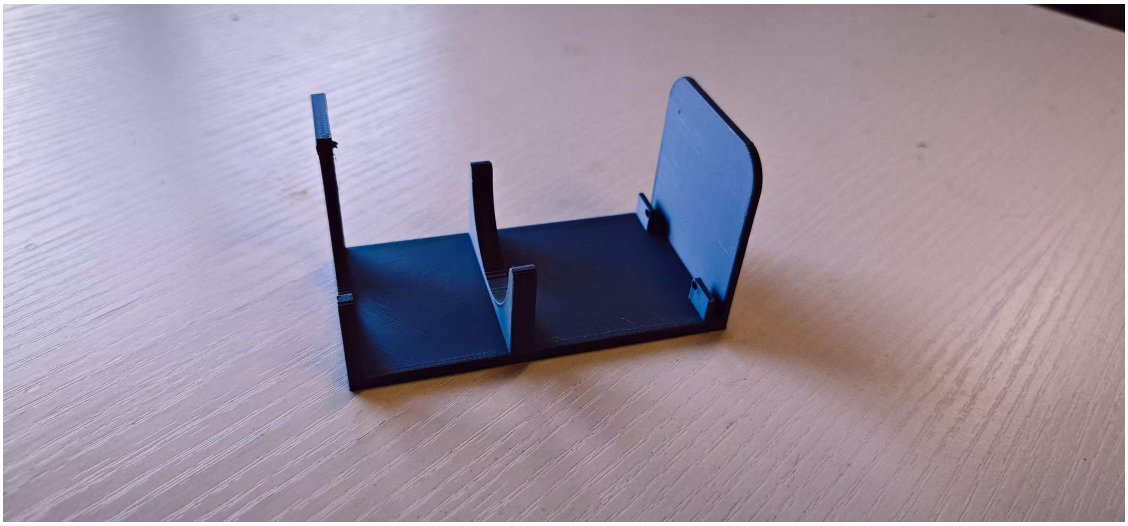
USB摄像模块 → USB PHY/PS USB Host → AR0234 Linux UVC采集

外部风扇降温

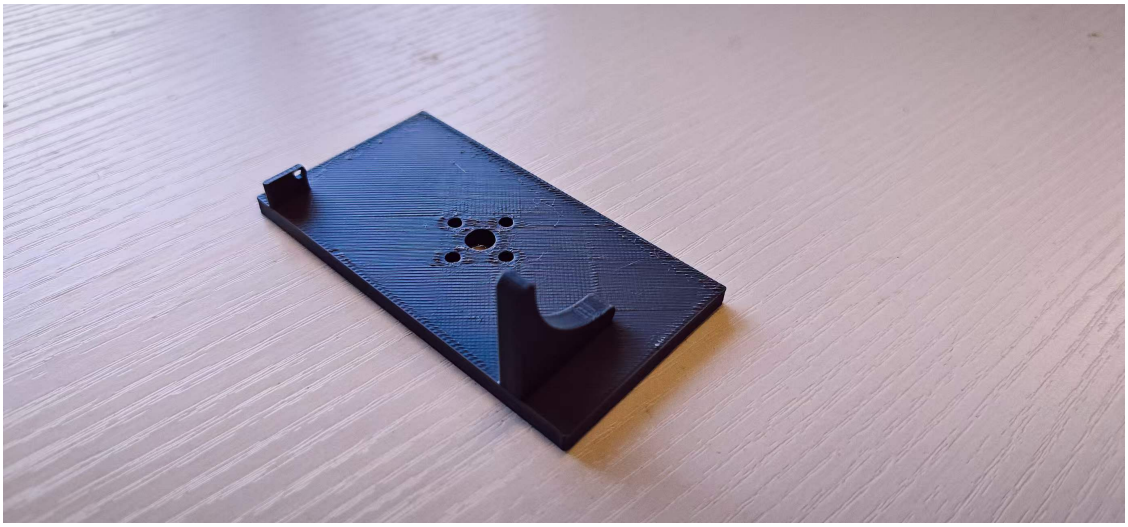
2.2.2机械结构介绍

机械结构分为3个部分：

1. 镜架结构，用于固定AR0234模块和100mm的镜头，防止模块和镜头抖动，并通过M4螺丝固定在云台上；

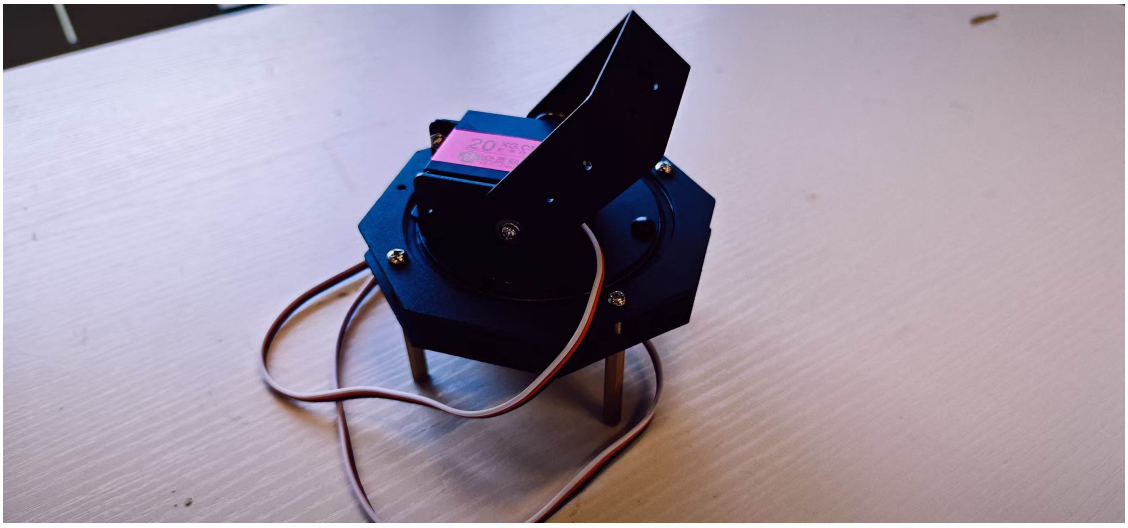


镜架结构



镜架结构图2

2. 云台结构，使用众灵科技PM20S舵机双轴云台：



PM20S舵机双轴云台

3. 箱子结构，除固定云台外，板卡和稳压模块贴在云台外，方便线的扭动.



底座箱子

2.2.3电路模块介绍

电路有3个模块：

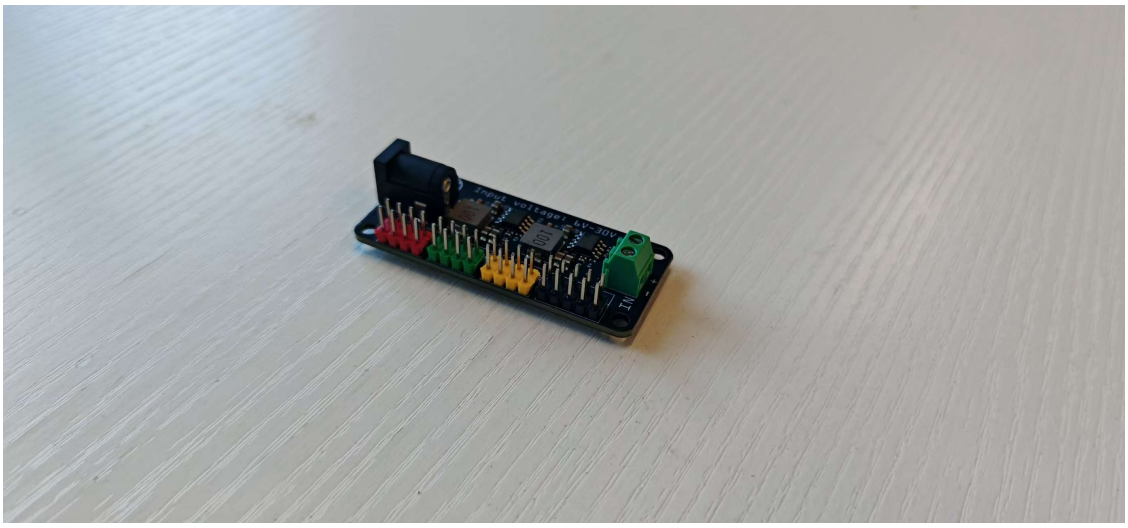
1. 电源线：电源加12V转5V稳压线路：

电源方面，通过DC接入稳压模块，稳压模块上再接两路舵机供电.



箱子内电池

稳压模块方面为12转5基本稳压模块：



稳压模块

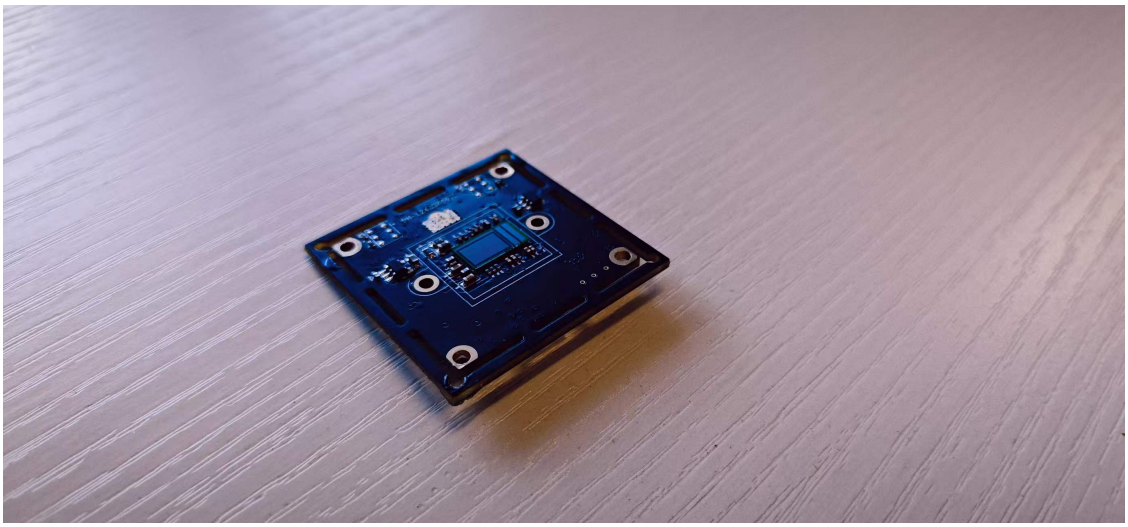
2. AR0234模块：100mm镜头加AR0234模块：

为满足“看的清”的要求，必须采取长焦镜头，我选用了100mm的镜头：



100mm长焦镜头

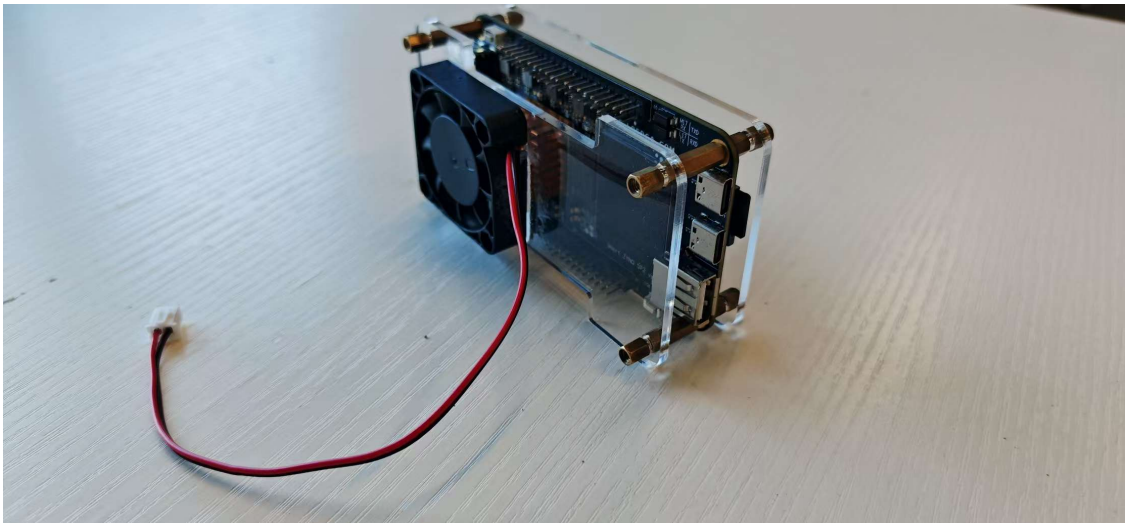
AR0234模块是一款由安美森设计的高性能CMOS数字图像传感器，有全局快门的优势，更容易看清高速飞行的鸟类，且功耗低，输出灵活，但由于滤波片原因，有些偏紫：



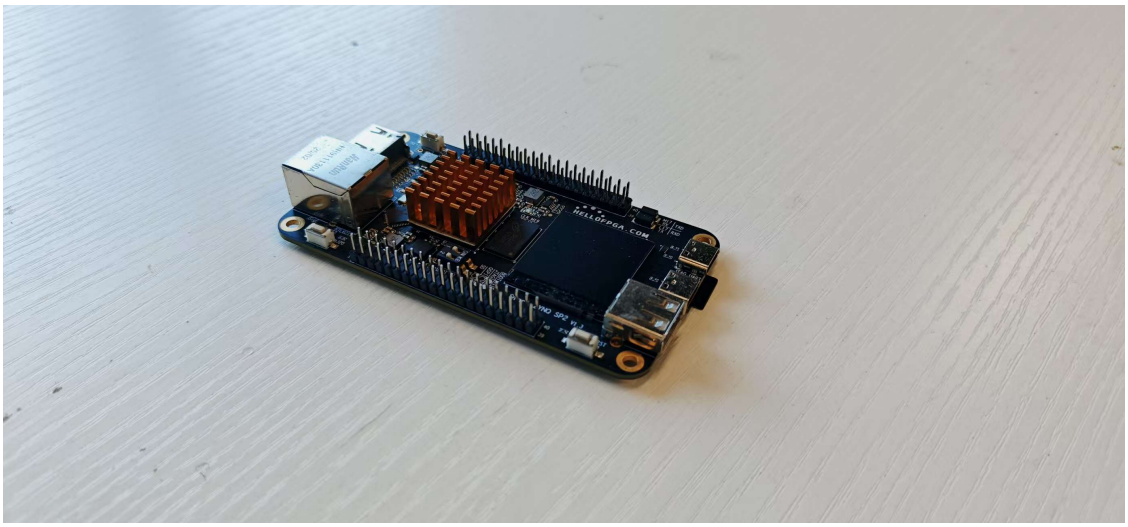
AR0234模块

3. xc7z020clg484-1 ZYNQ7020开发板：伟大，无需多言

我才用的是杭海电子的SP2开发板，体积小，成本低，方便布局与降低成本

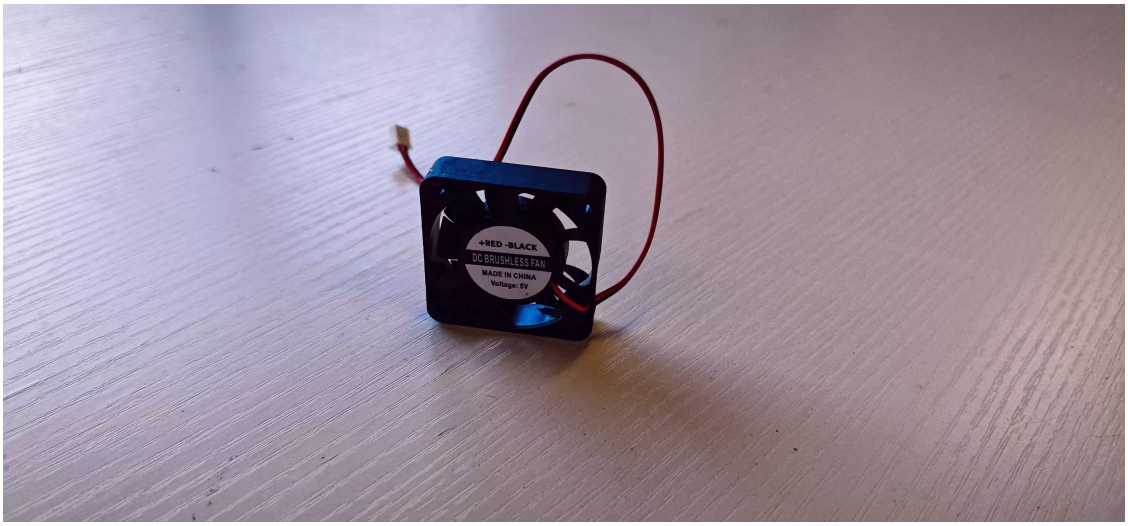


SP2开发板实物图1



SP2开发板实物图2

4. 外部小风扇



外部小风扇

2.3软件系统介绍

2.3.1软件整体介绍

如果接PC端会有软件，软件只有简单UI界面；不接则在PC端内没有，其余项目源码放在了文件包中，包含：

src：

--(vivado工程源码)

--(vitis工程源码)

--(最后选定模型)

UI：

--（UI界面代码和使用说明）

注：不包含xillinux，放在SD卡内，但src中有bit文件

2.3.2软件各模块介绍

第三部分_完成情况及性能参数

3.1整体介绍

完成情况比较良好，虽然未彻底实现实时处理，但有不错的资源利用和性能参数，部分参数如下：

1. 开发板端数据处理：

资源	已使用	总量	使用率	剩余
LUT	23,076	53,200	43.4%	30,124
FF	48,834	106,400	45.9%	57,566
BRAM tile	107	140	76.4%	33
DSP	216	220	98.2%	4

2. 模型卷积参数如下：

模型参数	数值
卷积数量	24 层，全部由 PL 执行
数值精度	22 层 INT8 输入/权重、INT32 累加；2 层浮点检测头采用 FP64 累加、FP32 输出
检测类别	单类别，当前软件按鸟类 class_id=0 处理
检测尺度	12×12、6×6，对应步长 16、32
检测位置	144＋36＝180
最终输出	5×180＝900 个 FP32 值，每位置为 x、y、w、h、score
DFL	每个边界方向 16 个分布区间
卷积权重数量	12,123,584 个，含 DFL 固定卷积
卷积权重＋偏置	12,128,179 项
参数文件	12,213,056 字节，约 11.65 MiB，含量化表及对齐填充
每帧卷积运算量	851,468,544 MAC，约 0.851 GMAC，按各层张量尺寸计算

24 层的具体尺寸如下，层号采用源码编号。所有卷积步长为 1；3×3 卷积 padding 为 1，1×1 卷积 padding 为 0。

层号	输出空间尺寸	输入→输出通道	卷积核
0	192×192	3→16	3×3
1	96×96	16→32	3×3
2	48×48	32→64	3×3
3	24×24	64→128	3×3
4	12×12	128→256	3×3

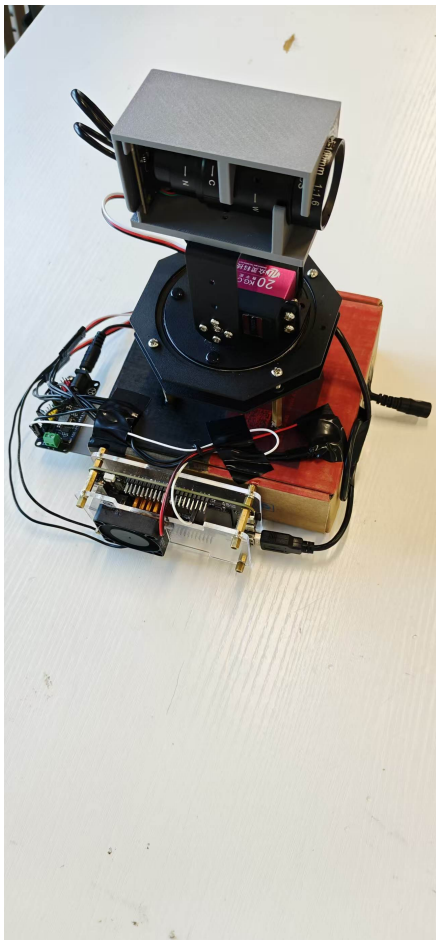
层号	输出空间尺寸	输入→输出通道	卷积核
5	6×6	256→512	3×3
6	6×6	512→1024	3×3
7	6×6	1024→256	1×1
8	6×6	256→128	1×1
9	6×6	256→512	3×3
10	6×6	512→64	3×3
11	6×6	512→256	3×3
12	12×12	384→256	3×3
13	6×6	64→64	3×3
14	6×6	256→256	3×3
15	12×12	256→64	3×3
16	12×12	256→256	3×3
17，浮点头	6×6	256→1	1×1
18	6×6	64→64	1×1
19	12×12	64→64	3×3
20	12×12	256→256	3×3
21，浮点头	12×12	256→1	1×1
22	12×12	64→64	1×1
23，DFL	4×180	16→1	1×1

3.2工程成果

工程成果完整介绍放在提交的视频中，这里也有链接：

3.2.1机械成果

完整机械成果连接组后如下：



系统整体画面

3.2.2电路成果

完整电路连接后成果如下，电路运行时正常

3.2.3软件成果

启动PC端UI界面，接受数据后，会显示预测数据如下：

3.3特性成果

重点讲述特性成果：

1. 硬件加速链路成果：

项目基于Zynq-7020构建“ARM预处理→共享DMA/DDR→HP0取数→并行卷积→重数量化→激活与池化→写回→ARM后处理”的加速链路，将原先由ARM处理的两个浮点检测头迁入PL，实现24层卷积全部硬件化。其中22层采用INT8运算与INT32累加，两个检测头保留精确浮点处理。计算阵列采用16输出通道并行、每路24项点积，并通过双INT8打包复用DSP。点积单元延迟15拍，流水填满后每拍可接受一组新输入；控制器与访存接口使用带缓冲的握手流水，保证背压下数据正确。最终使用216个DSP，在192×192输入下热帧推理为727毫秒，约1.38帧/秒，三组测试的72个卷积哈希全部匹配，形成了从计算结构、总线调度到板级数值校验的完整验证链路。

2. 超频成果：

项目以初始100MHz为基准，逐步经过170MHz、200MHz验证，最终将PL工作频率提升至240MHz，达到原来的2.4倍。为支撑提频，RTL开启DSP输入、乘法和输出寄存器，将宽加法拆为两拍分段进位，细化点积归约树，并对控制器与HP0之间的握手路径进行流水隔离。同时前移地址边界比较，分组处理高扇出使能和同步复位，降低组合逻辑与布线延迟。最后一轮修复级联BUFG问题，保持MMCM输出直接驱动PL时钟缓冲，总BUFG为3个，并保持原始时序约束。最终建立余量为正0.067纳秒，保持余量为正0.020纳秒，板端确认240MHz配置，8次生产推理全部通过。该成果体现频率提升及真实时序收敛能力，端到端吞吐仍以板测为准。

3. 片上缓存与写合并成果：

针对卷积重复取数和零散写回，项目采用128KiB输入缓存与192KiB分组权重存储。输入缓存划分为八个银行，权重按输出通道组织并跨像素复用，减少同一数据从DDR反复读取。访存通过64位HP0接口，使用64字节缓存行和1至8拍突发填充，并检查响应、事务标识及结束标记，完整接收后才发布有效缓存标签。BRAM采用同步读及输出寄存，将存储器访问与后级逻辑分隔，以支撑240MHz运行。写回侧把同一64位字中的多个8位结果合并，以字节掩码标识有效位置，在字边界或收尾时提交，减少细粒度写请求与响应开销。最终BRAM占用107/140，保留33个存储块，实现了容量、访问复用和物理时序之间的平衡。

4. 模型跳零和算子融合与数据流优化结合：

项目把模型激活跳零、算子融合与访存调度统一设计。控制器以24个激活为一块，组包时逐字节累计非零标志；全零块跳过对应乘加，非零块通过映射表保留原始块编号，确保压缩后仍能读取正确权重。全零输入组仍执行偏置及后续量化处理，保持计算语义。卷积累加结果直接衔接重数量化、激活查表和可选池化，再进入带字节掩码的写合并单元，减少子算子之间的中间张量落存。重数量化采用完整有效乘积、偶数舍入及饱和处理，数据与通道标记同步流水；请求和响应在背压时按握手协议保存。该组合使稀疏计算节省、片上结果复用与DDR事务减少相互配合，并通过逐层板测核对正确性；实际加速收益随激活稀疏程度、数据复用和访存等待变化。

第四部分_总结

4.1可拓展处

展示中，我使用了USB JTAG连接开发板和PC端，用于传输数据和供电。实际运用时，可以设计RJ45网口，应用检测结果 → UDP/IP → 以太网MAC → RGMII → PHY → 磁性器件/RJ45 → 网络，传输数据，再使用5V电源单独对板子进行供电。如声光驱鸟装置等驱鸟装置，实现完整系统。也可加入天线，通过无线传输数据至服务器，处理数据。

4.2心得体会

这是我第一次进行嵌入式开发，去完整的进行一次嵌入式项目，还是基于FPGA的，这很难，但也很值得。一开始我想的是去做一个类似课设之类的东西，比如千兆网UDP回环之类的。一次偶然的机会，我在学校看到了飞鸟，想到了人与自然和谐共生的时代问题，人应该如何去与自然相处，便决定去做一个有实际意义的，符合时代要求的项目。选来选去，结合FPGA的并行硬件优势，我走上了FPGA图像检测的经典道路：实现YOLO算法在FPGA上的部署，解决鸟类识别问题。

我的资源很有限，由于考虑完整实现功能和成本等重要因素，我明确了使用ZYNQ7020去实现YOLO算法的方向，并把工程地位定义为一个DEMO，帮助有更高成本与资源项目去更好的开发，为比如使用ZU系列去实现实时检测等等项目进行开发，这是我觉得我去利用大部分xc7z020clg484-1资源可以做到的事。

定位的层级让我把这项工程想象得很简单，但是实际工程遇到的困难打得我错手不急：一开始得YOLOV8模型部署完全超出了我和我的开发板的能力，面对严重的逻辑资源短缺和计算带宽与片上存储瓶颈，我不得不舍弃这种方案；我看到了一篇很早的开罗大学的文章，他们团队使用了LENET5 CNN神经网络，实现了手写数字的识别，除了LENET5还有很多种不同版本的，24层卷积，16层卷积.....我参考了他们的方案进行设计，但最后在2018.3Vivado中IO资源的分配，仿真，和blockdesign中的连接改变频率时出现了极大的问题。最后我选择了Tinyyolov3yu的方案。

模型训练上，我沿用了YOLOV8的数据集进行训练，Tinyyolov3yu沿用了YOLOV8的检测头。我在自己的电脑上训练了4155张，val819张，最后的训练结果为后续部署提供了基础。但随后的验证和优化中，出现了大量数据一致性检验、冒烟测试、建立时间违例和BRAM资源超限等问题。整个10月的工作集中在数值核对、资源调整与时序优化，已保留基线版本的板级验证记录，单次推理约727ms，约1.38FPS。训练端的精度结果不能直接作为192×192板端精度结论，本工程尚未完成该输入尺寸下的标注测试集精度测量。留有遗憾，但能在各种阻力下完成这个项目，让我获得了一次深入学习嵌入式视觉设计的机会。

下面主要讲一讲碰到的工程要点问题：

第一，重点讲一下时序约束问题。时序约束是超频加速中相当大的一个阻碍，从一开始就不断有SETUP的负裕量的error和warning，综合布线失败了很多次。**时序问题**的本质是240MHz下每拍仅4.167ns，寄存器输出、组合计算、布线和建立时间必须满足周期预算，同时保持检查则限制数据不能过早到达。我之前的关键路径布线占77.8%，瓶颈在于长进位链和控制信号的大扇出。我通过以下几个方面来进行时序约束，来进行满足要求的综合布线：首先，我进行了流水线处理来缓解BRAM和时序问题，我在核心24项点积单元采用了15级流水线，结合16路输出通道并行。在240MHZ的频率下，时间只需62.5ns，持续供数时，每拍可以接收一组新向量。在重数量化上，采取14级流水线处理；控制器与HP0上，设计了请求方向二级，响应方向二级。

其次，我把计算侧开启DSP内部寄存器，把五层加法树各拆为两拍：先独立计算高低部分，再合并低位进位，形成15级点积流水，最后将关键支路LOW_WIDTH由8改7。控制侧方面为HPO请求、响应各加两级带缓冲握手流水，切断跨模块长路径。VALID与数据方面同步，背压时保存事务。物理侧层面保留局部激活副本、分配复位负载、寄存BRAM输出，缩短连线并降低负载，然后HPO边界比较前移到已有等待周期。

最后，修复级联BUFG，使MMCM直接驱动PL时钟缓冲。在原始约束下完成布局布线，建立、保持余量分别为+0.067ns、+0.020ns。该时序修正版已完成实现检查，完整板级数值核对记录对应单独保留的基线版本；两者的位流与验证记录分别保存。这一步我的核心处理原则是减少每拍必须完成的工作和物理传播距离，并保持事务与数值语义一致。

第二，讲一讲管脚约束：

- 串口：RX绑定M17、TX绑定L17，采用LVCMOS33，匹配板载3.3V电平。因为串口属于异步PS外设，对其设置false_path，避免误按PL同步时钟检查。
- USB摄像头：PS USB0使用MIO28～39，复位使用MIO46，匹配板载USB PHY连接。
- DDR与时钟：DDR沿用PS专用管脚配置；PL的240MHz由片内MMCM产生，所以没有新增外部时钟管脚。

第三，介绍随项目提交的ROSS RTL流水线skill。该扩展面向Vivado 2025.2的RTL设计与验证流程，将资源预算、端口带宽、频率、流水级数、启动间隔和数值语义整理为明确的设计约束，并保留中文批注、语义命名与可维护的模块结构。2.2.0版本提供资源规划与时序评估脚本，以及缓冲、乘加、求和树、分叉和舍入饱和五类RTL示例，配套独立自检与参数化验证记录。其目标是让设计、仿真和实现结果可以逐项核对，而不是仅凭代码外观判断效果。新舍入示例尚未测量布线后的最高频率或板上性能；已有200MHz完整基准仍存在输出路径违例，这些边界随验证记录一并保留。该扩展是社区兼容版本，许可证与参考资料来源见提交目录中的skill文件夹。

在文末，我想说，再次感谢这次比赛给我机会去学习这门有趣的设计课程。我领会到了一次优秀的、极致的嵌入式设计并不简单，我还有许多东西去见识，许多经验去积累。希望在未来，我能把数字系统设计做到炉火纯青，去实现一些更加极限和好玩的想法。

第五部分_参考文献

[1] Abdulkader A, Eskandar D, Essam O, et al. Implementation of a convolutional neural network on an FPGA using Verilog[R/OL]. https://github.com/omarelhedaby/CNN-FPGA/blob/master/Research%20Report.pdf.